MLOps Learning Summary

This document summarizes the major MLOps concepts implemented in the student-performance project. The project evolved from a simple ML training script into a production-aware MLOps pipeline with monitoring, validation, deployment safety, infrastructure automation, and feature management.

1. DVC (Data Version Control)

DVC was used to version datasets, parameters, metrics, and machine learning outputs. Instead of manually tracking model files and datasets, DVC creates reproducible pipelines. What was learned: • Versioning datasets • Tracking metrics.json • Managing model artifacts • Using params.yaml for hyperparameters • Reproducible ML pipelines Example: training.py reads parameters from params.yaml: learning_rate = 0.1 max_depth = 4 DVC ensures that when parameters change, the entire training pipeline can be reproduced exactly.

---

2. Evidently AI (Monitoring & Drift Detection)

Evidently AI was used to monitor data drift and production performance. What was implemented: • Drift detection • HTML monitoring reports • JSON report generation • Automatic retraining triggers • Smart retraining logic • Performance degradation checks • Cooldown-safe retraining • Rollback-safe deployment logic Example: If production data suddenly changes: studytime = 500 absences = 100 Evidently compares: Reference Data vs Current Production Data If drift score becomes high: • Alert generated • Retraining triggered • Performance checked • Rollback possible Smart retraining condition: Retrain only if: 1. Drift is high 2. Production performance is bad This prevents unnecessary retraining loops.

---

3. Terraform (Infrastructure as Code)

Terraform was learned for automating cloud infrastructure. What Terraform solves: Instead of manually creating infrastructure on cloud providers, Terraform creates everything using code. What was learned: • Infrastructure as Code (IaC) • EC2 instance provisioning • Security group creation • Reproducible infrastructure • Automated deployments Example: A Terraform file can automatically create: • Virtual machines • Network rules • Cloud storage • Kubernetes clusters Example command: terraform apply This deploys infrastructure automatically.

---

4. Helm Charts (Kubernetes Package Management)

Helm Charts were learned to simplify Kubernetes deployments. What Helm solves: Kubernetes YAML files become massive and difficult to manage. Helm converts them into reusable templates. What was learned: • values.yaml • deployment templating • Kubernetes package management • Reusable deployments • Version-controlled deployments Example: Instead of editing deployment.yaml manually, values.yaml controls: • image version • replica count • ports • environment variables Example: replicaCount: 3 image: repository: madhug17/ml-fastapi-app Helm dynamically generates Kubernetes manifests from templates.

5. Feast Feature Store (Upcoming Learning Topics)

Feast is the next major MLOps component. Why Feast matters: Training pipelines and production APIs often use different feature logic. This creates training-serving skew. Feast solves this by acting as a centralized feature store. Topics planned for learning: • Feature Views • Entities • Offline Store • Online Store • Point-in-time correctness • Historical feature retrieval • Real-time feature serving • Training-serving consistency • Feature freshness • Feature TTL • Batch features • Online inference features Example: Instead of reading features directly from CSV: pd.read_csv("student-mat.csv") Production system will fetch features from Feast: store.get_online_features(...) This guarantees: • Training uses same features • Production uses same features • No feature mismatch occurs

---

6. Production-Level MLOps Pipeline Built

Final capabilities implemented in the project: • MLflow experiment tracking • DVC pipeline tracking • Automated retraining • Data drift monitoring • Production performance monitoring • Safe deployment validation • Model versioning • Rollback system • Cooldown-safe retraining • Logging system • HTML monitoring dashboards • Kubernetes deployment • Helm chart management • Infrastructure automation Final architecture: User Request ↓ FastAPI Service ↓ Feature Store (Feast) ↓ Production Model ↓ Prediction Logging ↓ Evidently Monitoring ↓ Drift Detection ↓ Retraining Pipeline ↓ Safe Model Promotion

---